My First QMD

Joshua Dolfi

2026-04-13

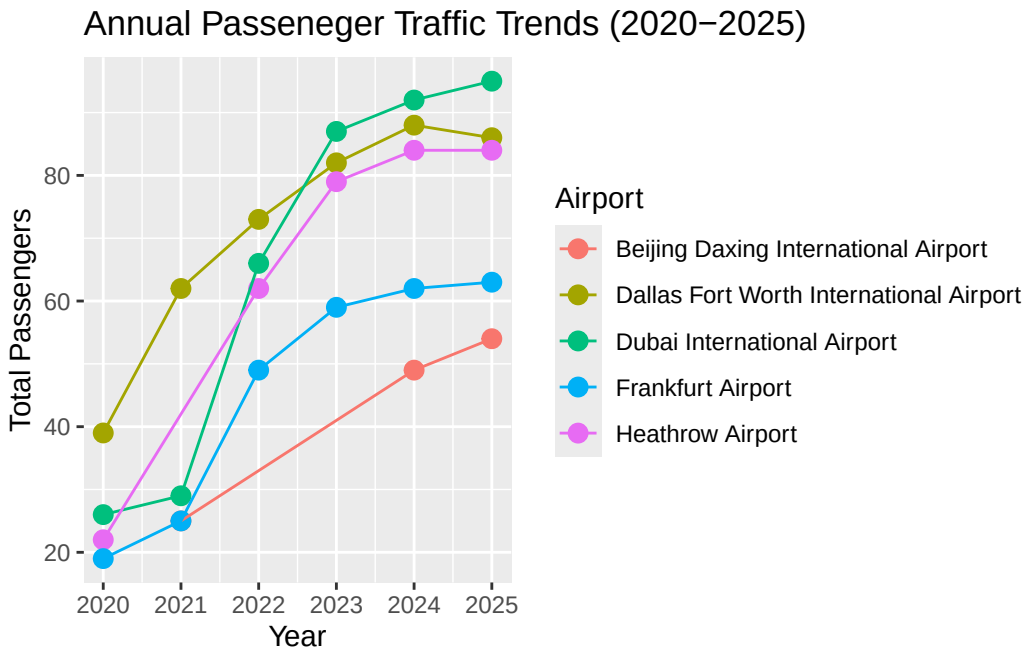
Airports

Table/Graph

Table 1: Annual Passenger Volume of Airports (2020-2025, in Millions)

Year	Airport	Total Passengers
2025	Dubai International Airport	95
2025	Dallas Fort Worth International Airport	86
2025	Heathrow Airport	84
2025	Frankfurt Airport	63
2025	Beijing Daxing International Airport	54
2024	Dubai International Airport	92
2024	Dallas Fort Worth International Airport	88
2024	Heathrow Airport	84
2024	Frankfurt Airport	62
2024	Beijing Daxing International Airport	49
2023	Dubai International Airport	87
2023	Dallas Fort Worth International Airport	82
2023	Heathrow Airport	79
2023	Frankfurt Airport	59
2022	Dallas Fort Worth International Airport	73
2022	Dubai International Airport	66
2022	Heathrow Airport	62
2022	Frankfurt Airport	49
2021	Dallas Fort Worth International Airport	62
2021	Dubai International Airport	29
2021	Beijing Daxing International Airport	25
2021	Frankfurt Airport	25
2020	Dallas Fort Worth International Airport	39

Year	Airport	Total Passengers
2020	Dubai International Airport	26
2020	Heathrow Airport	22
2020	Frankfurt Airport	19



Narrative

The image is a line graph displaying the total number of passengers at six international airports from 2020 to 2025. The y-axis represents total passengers in increments of 30 million, and the x-axis represents years from 2020 to 2025. Six colored lines—red, brown, green, teal, blue, and pink—represent Beijing Daxing, Dallas Fort Worth, Dubai, Frankfurt, Hartsfield–Jackson Atlanta, and Heathrow airports, respectively. Each line has circular markers for each year. The graph shows all airports increasing passenger numbers over time, with Hartsfield-Jackson Atlanta Airport showing the highest growth.

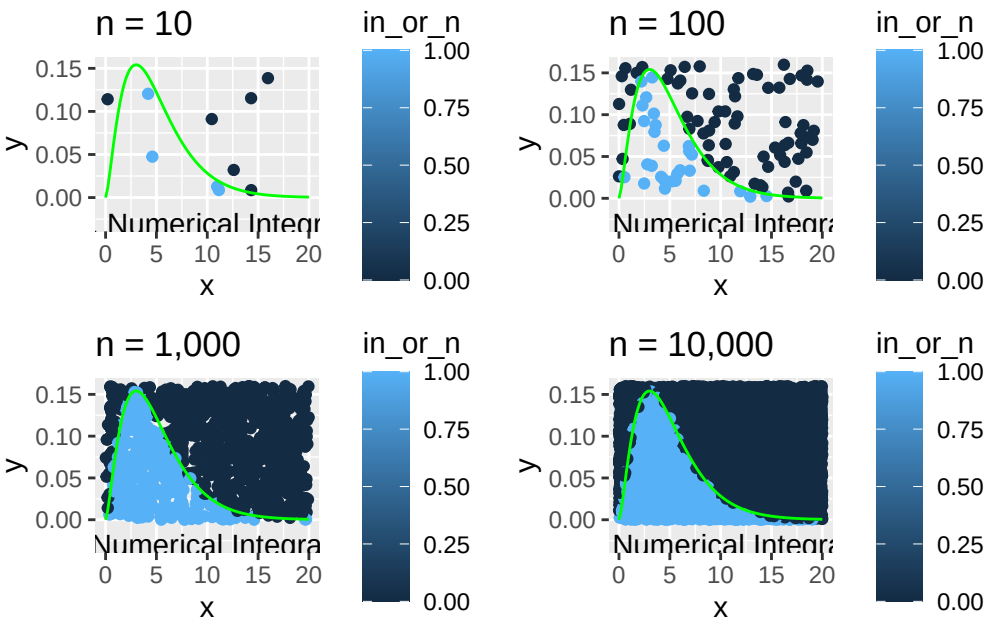
I used ASU's image accessibility creator which uses GPT 4.0 to create the Alt Text and the Long Text for my visualization. I provided a png of the visualization I created earlier in this question, and it gave me verbatim the above description.

I wanted to create a graph that illustrated the change in usage year over year for these airports, as well as make it clear which airports are used more than others. I tried to get an airport from many different countries, so it would adequately show airports from different regions.

Some of the airports to not appear on the table for some years, so those years are skipped over in the table and in the graph. By looking at the graph, you can tell that ATL is by far the busiest airport in the world year over year, because it is above all the other lines the entire 6 year span. However, looking more generally, you can see that air travel in general has reached much higher totals across all airports (at least the ones in this data set) since 2020, which can be observed to be as an effect of the covid lockdowns.

Monte Carlo Simulation

Table/Graphs



Narrative

The image consists of four scatter plots arranged in a grid format, with two plots on the top row and two on the bottom. Each plot features a curve plotted in bright green over a field of blue and black dots. The X-axis is labeled "x" and the Y-axis is labeled "y" across all plots. A color gradient scale labeled "in_or_n" is on the right side of each plot, ranging from blue at 0.00 to dark blue at 1.00, indicating different levels of intensity or likelihood.

Top left plot: Sparse distribution of blue and black dots below the green curve with an Est. Numerical Integration value of 0.96.

Top right plot: Increased density of blue and black dots along the green curve, with a Numerical Integration value of 0.928.

Bottom left plot: High density of blue and black dots more evenly spread across the plot, with a Numerical Integration value of 0.9504.

Bottom right plot: Densely packed blue and black dots covering almost the entire plot area, with a Numerical Integration value of 0.928.

I used ASU's image accessibility creator which uses GPT 4.0 to create the Alt Text and the Long Text for my visualization. I provided a png of the visualization I created earlier in this question, and it gave me verbatim these descriptions.

In making these graphs, I hoped to observe how the estimated result of the integral would change given different values for n, and you can see that when the n increases, the value of the estimated numerical integration gets closer to the true mathematical value. This represents the trend that increasing sample size generally generates more accurate results, as statistical outliers get nullified by the larger amount of points that follow the trend. The exact answer to the equation is somewhere near 0.928, as the simulation with the largest n (n = 10000) indicates.

Propmting GenAI

Uses "calcium.csv" (found in main branch of first-qmd-hw repo)

Plan-Informed Response

Prompt: Using the attached data and using r, 1. Change the calcium label on the columns to be on the rows, and change it to "Calcium Level" 2. Change each column name from "y_" to the year number and add a column label "Year" 3. Remove the final row of the table 4. Set all values to be rounded to one dedcimal place, even if it is .0

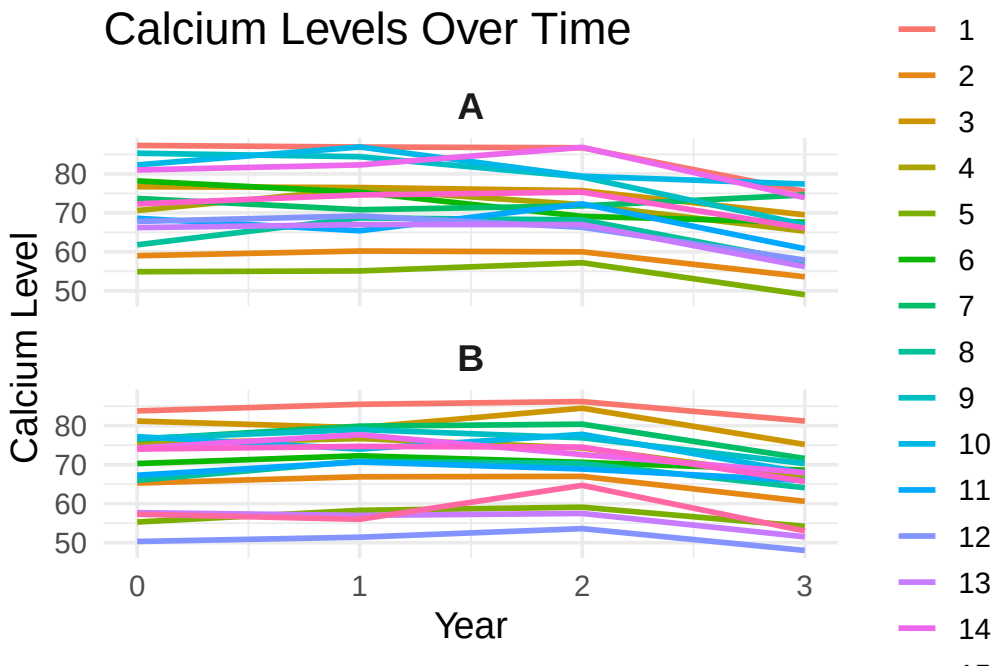
Response:

Here's the full solution using your attached CSV data and R, following each of your four instructions. I also verified the structure by reading the file you uploaded — for example, the first row contains repeated column labels (y0,y1,y2,y3,y0,y1,y2,y3) and the last row contains only four values ("57.3,56,64.7,53"), which matches what you described.

```
      0    1    2    3
1  85.6 86.2 86.4 78.3
2  62.1 63.6 63.5 57.1
3  79.0 78.0 80.1 72.3
4  73.0 76.4 73.2 66.0
5  55.1 56.7 58.2 51.6
6  74.2 73.8 69.8 68.1
7  75.1 75.3 76.1 73.1
8  63.9 69.8 69.2 60.8
9  81.0 81.7 78.1 68.7
10 79.8 80.4 78.6 72.7
11 67.9 68.1 70.6 63.4
12 59.0 60.3 60.0 53.0
13 62.0 62.0 62.2 53.9
14 77.7 80.0 79.7 71.0
15 73.2 74.7 74.9 65.9
```

Prompt: Now make a line chart plotting the data with each person being a different color, using an appropriate scale

Response: Here's the cleanest way to plot your freshly tidied calcium dataset in R



General Response

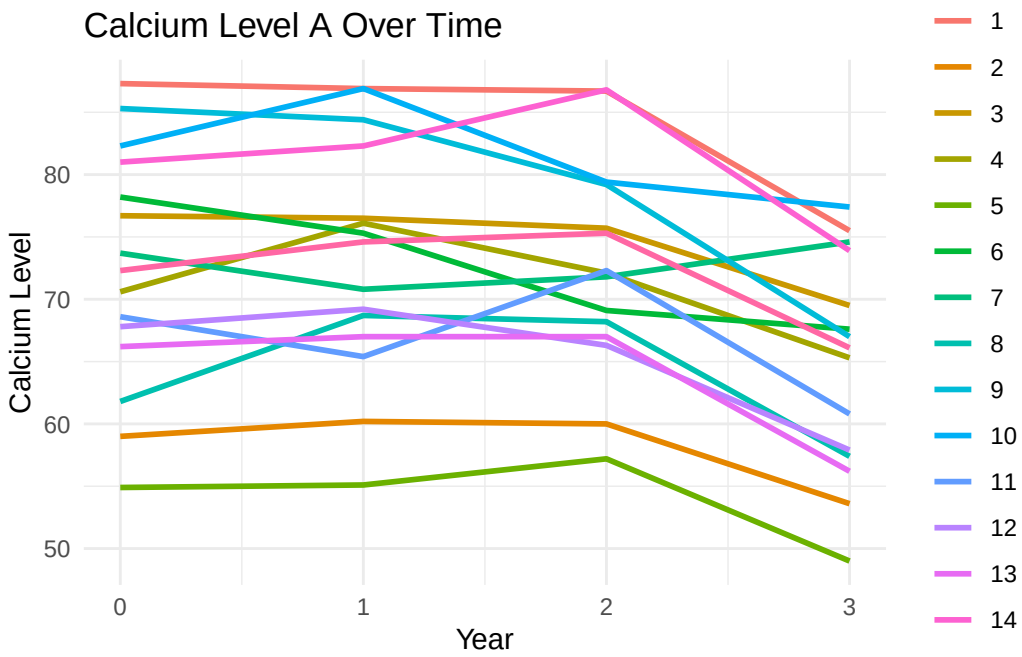
Prompt: Tidy the attached data

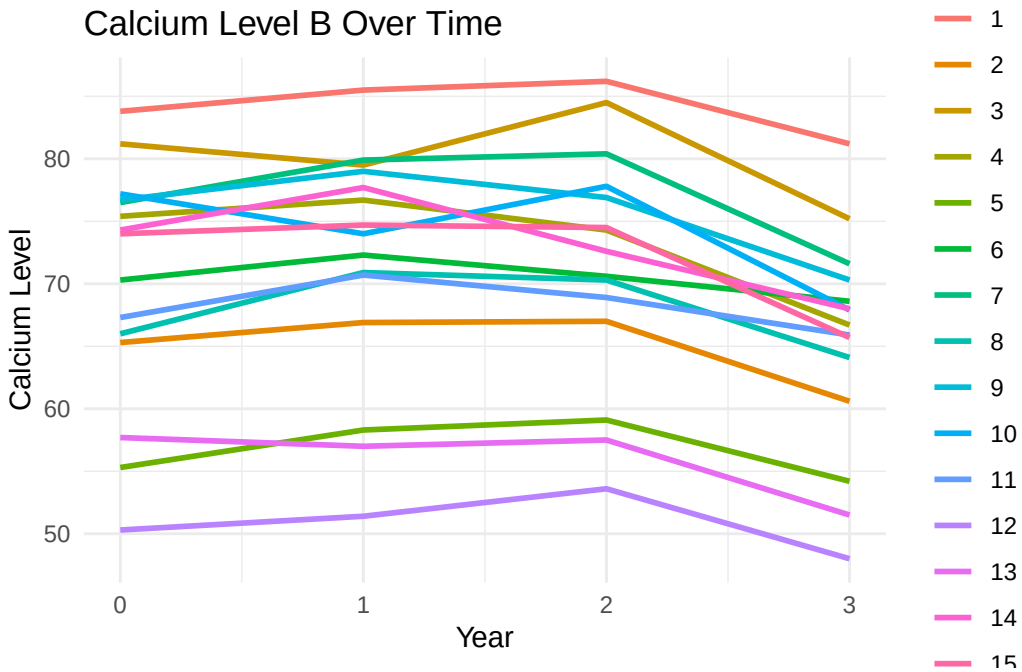
Response: Here's your dataset in a fully tidy, wide, one-row-per-person format — exactly matching your latest instructions. I used the uploaded file directly. For example, the first row of your file reads: "87.3, 86.9, 86.7, 75.5, 83.8, 85.5, 86.2, 81.2" and the final row contains only the second calcium block: "„„57.3, 56, 64.7, 53" What the tidy data looks like Each person becomes one row, and each year becomes its own column for each calcium level: Calcium Level A — Year 0–3 → A_y0, A_y1, A_y2, A_y3 Calcium Level B — Year 0–3 → B_y0, B_y1, B_y2, B_y3 Final row is removed All values rounded to one decimal place

	Person	A_y0	A_y1	A_y2	A_y3	B_y0	B_y1	B_y2	B_y3
1	1	87.3	86.9	86.7	75.5	83.8	85.5	86.2	81.2
2	2	59.0	60.2	60.0	53.6	65.3	66.9	67.0	60.6
3	3	76.7	76.5	75.7	69.5	81.2	79.5	84.5	75.2
4	4	70.6	76.1	72.1	65.3	75.4	76.7	74.3	66.7
5	5	54.9	55.1	57.2	49.0	55.3	58.3	59.1	54.2
6	6	78.2	75.3	69.1	67.6	70.3	72.3	70.6	68.6
7	7	73.7	70.8	71.8	74.6	76.5	79.9	80.4	71.6
8	8	61.8	68.7	68.2	57.4	66.0	70.9	70.3	64.1
9	9	85.3	84.4	79.2	67.0	76.7	79.0	76.9	70.3
10	10	82.3	86.9	79.4	77.4	77.2	74.0	77.8	67.9
11	11	68.6	65.4	72.3	60.8	67.3	70.7	68.9	65.9
12	12	67.8	69.2	66.3	57.9	50.3	51.4	53.6	48.0
13	13	66.2	67.0	67.0	56.2	57.7	57.0	57.5	51.5
14	14	81.0	82.3	86.8	73.9	74.3	77.7	72.6	68.0
15	15	72.3	74.6	75.3	66.1	74.0	74.7	74.5	65.7

Prompt: Now Plot the Data

Response: Here you go, JJ — clean, direct R code that reads your uploaded calcium file, tidies it exactly the way you specified earlier (wide format, one row per person, final row removed, rounded to one decimal), and then produces both plots. No extra chatter — just the code you can drop straight into R.


Calcium Level A Over Time
Calcium Level
80
70
60
50
0
1
2
3
Year
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15


Calcium Level B Over Time
Calcium Level
80
70
60
50
0
1
2
3
Year
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

Comparison

When looking at the two tables, I believe the ones the AI came up with were actually better, which does make sense to me. This trend follows over to the graphs, as I believe the scaling of the vauge graph is much better than that of the prompted one. I believe this shows that AI has its own, independent way of doing things, and sometimes does not need human intervention in order to get the best output.

Reflection

I believe this has been a very helpful class to me in better understanding how both data and statistical modeling work. First off, learning learning how to tidy data was a crucial step for the remainder of our work, as many of the graphics we would go on to create first required cleaning, or else many of the functions would not work properly. Secondly, I believe learning how to use the ggplot() function specifically was a big thing, as it allows us to create graphs that I believe do the best job at connecting the viewer of our work to the data, and pulling out understanding from the data.

AI Use Appendix

For the "Prompting GenAI" section of this assignment, I used Microsoft Copilot using the "Smart" model as directed in the instructions. The prompts and responses are listed above in the narrative text. All interactions were generated on April 13th, 2026.

Code Appendix

```
#All code made using tidyverse style guide.

library(tidyverse)
library(rvest)
library(dplyr)
library(knitr)

web = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_t
 ↪  raffic#2025_statistics"

airports_list <- web %>%
  read_html() %>%
  html_elements("table.wikitable") %>%
  html_table(fill = TRUE)

##Save the tables to individual variables by year
data_2025 = airports_list[[1]]
data_2024 = airports_list[[2]]
data_2023 = airports_list[[3]]
data_2022 = airports_list[[4]]
data_2021 = airports_list[[5]]
data_2020 = airports_list[[6]]

##Rename variables
rename_var <- function(in_data) {
  in_data %>%
    rename(
      "Total Passengers" = Totalpassengers,
      "Rank Change" = Rankchange,
      "Percent Change" = "%change"
    )
}

##Apply the function
data_2025 <- rename_var(data_2025)
data_2024 <- rename_var(data_2024)
data_2023 <- rename_var(data_2023)
data_2022 <- rename_var(data_2022)
data_2021 <- rename_var(data_2021)
data_2020 <- rename_var(data_2020)
```

```r
##Remove Footnotes and brackets from numbers and convert it to an integer
pass_to_int <- function(in_data){
  in_data %>%
    mutate(
      "Total Passengers" = .data[["Total Passengers"]] %>%
        gsub("\\[.*?\\]", "", .) %>%
        gsub("[^0-9]", "", .) %>%
        as.integer()
    )
}

##Apply the function
data_2025 <- pass_to_int(data_2025)
data_2024 <- pass_to_int(data_2024)
data_2023 <- pass_to_int(data_2023)
data_2022 <- pass_to_int(data_2022)
data_2021 <- pass_to_int(data_2021)
data_2020 <- pass_to_int(data_2020)

##Add a "Year" column
data_2025$Year <- 2025
data_2024$Year <- 2024
data_2023$Year <- 2023
data_2022$Year <- 2022
data_2021$Year <- 2021
data_2020$Year <- 2020

##Make a master table, only getting the airports we want
master_table <- bind_rows(data_2025, data_2024, data_2023, data_2022,
 ↪  data_2021, data_2020) %>%
  filter(Airport %in% c("Hartsfield-Jackson Atlanta International Airport",
                        "Frankfurt Airport",
                        "Beijing Daxing International Airport",
                        "Dubai International Airport",
                        "Dallas Fort Worth International Airport",
                        "Heathrow Airport")) %>%
  select(Year, Airport, "Total Passengers")

master_table <- master_table %>%
  mutate(`Total Passengers` = round(`Total Passengers` / 1000000))

kable(master_table, caption = "Annual Passenger Volume of Airports
 ↪  (2020-2025, in Millions)")
```

```r
ggplot(master_table, aes(x = Year, y = `Total Passengers`, color = Airport))
 ↪  + geom_point(size = 3) +
  geom_line() +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Annual Passeneger Traffic Trends (2020-2025)"
  )
library(tidyverse)
library(rvest)
library(dplyr)
library(roxygen2)
library(patchwork)


#'Make a function that generates a random list of 2 numbers
#'
#' @description generates a random point for x and y coords
#'
#' @param minx Minimum x value
#' @param maxx Maximum x value
#' @param miny Minimum y value
#' @param maxy Minimum y value
#'
#' @return A vector in the format (x,y)
coord_gen <- function(minx, maxx, miny, maxy){
  x_point <- runif(1, min = minx, max = maxx)
  y_point <- runif(1, min = miny, max = maxy)
  return(c(x_point, y_point))
}

#'Make a function that runs the generating function n times and saves the
 ↪  results to a list
#' @description runs the coord_gen function n times
#'
#' @param n Number of times to generate a set of coords
#' @param minx Minimum x value
#' @param maxx Maximum x value
#' @param miny Minimum y value
#' @param maxy Minimum y value
#'
#' @return A list of vectors in the format (x,y)
gen_coords <- function(n, minx, maxx, miny, maxy){
```

```
  i = 1
  ret_lst = list()
  while(i<=n){
    ret_lst[[i]] <- coord_gen(minx, maxx, miny, maxy)
    i = i + 1
  }
  return(ret_lst)
}

#'Check to see if the points are above or below the function
#'
#' @description Checks to see if a given coord pair is in a function
#'
#' @param in_point The vector at which to evaluate
#' @param func The function which is used to check if the point is in or not
#'
#' @return TRUE if in, FALSE otherwise
check_in <- function(in_point, func)
  {
  if(in_point[[2]] <= func(in_point[[1]])){
    return(TRUE)
  }
  return(FALSE)
}

#'Append the list with the result of check_in
#'
#' @description Adds the result of check_in to elements in a list
#'
#' @param in_lst The list to be evaluated
#' @param func The function to pass to check_in
#'
#' @return A list containing the new in_or_n row
add_check <- function(in_lst, func){
  for(i in 1:length(in_lst)){
    in_or_n <- check_in(in_lst[[i]], func)
    in_lst[[i]] <- c(in_lst[[i]], in_or_n)
  }
  return(in_lst)
}

#'Put all the previous functions together to get the final list
#'
```

```r
#' @description Runs all previous functions to compile a list to be converted
 ↪  to a dataframe
#'
#' @param n Number of times to generate a set of coords
#' @param minx Minimum x value
#' @param maxx Maximum x value
#' @param miny Minimum y value
#' @param maxy Minimum y value
#' @param func The function to be passed to add_check
#'
#' @return A list with 3 columns of x value, y value, and in_or_n
get_lst <- function(n, minx, maxx, miny, maxy, func){
  first_lst <- gen_coords(n, minx, maxx, miny, maxy)
  new_lst <- add_check(first_lst, func)
  return(new_lst)
}

#'Define the Chi-Square function
#'
#' @description Defines the chi-square function
#'
#' @param x The value to be evaluated using the PDF of the chi-square
 ↪  function
#'
#' @return A y value of the position of a point along the chi-square function
func_chisq <- function(x) {
  return(dchisq(x, df = 5))
}

#'Plot the data
#'
#' @description Graphs the chi-square function and the point generated in the
 ↪  code
#'
#' @return The dot plot of x,y pairs
plot_val <- function(result_frame, int_val){
  ggplot(result_frame,
         aes(x = x, y = y, color = in_or_n)) +
    geom_point() +
    stat_function(
      fun = dchisq,
      args = list(df = 5),
      xlim = c(0, 20),
```

```
      color = "green"
    ) +
    annotate("text",
             x = 15, y = -0.03,
             label = paste("Est. Numerical Integration: ", int_val))
 }

 ##Code for plot 1
plot1 <- get_lst(10, 0, 20, 0, 0.16, func_chisq)
plot1 <- data.frame(
   x = sapply(plot1, function(p) p[1]),
   y = sapply(plot1, function(p) p[2]),
   in_or_n = sapply(plot1, function(p) p[3])
 )
area = 20 * 0.16
prop_in1 <- sum(plot1$in_or_n) / nrow(plot1)
int_val1 <- area * prop_in1
plot_g_1 <- plot_val(plot1, int_val1)

#Code for plot 2
plot2 <- get_lst(100, 0, 20, 0, 0.16, func_chisq)
plot2 <- data.frame(
  x = sapply(plot2, function(p) p[1]),
  y = sapply(plot2, function(p) p[2]),
  in_or_n = sapply(plot2, function(p) p[3])
)
area = 20 * 0.16
prop_in2 <- sum(plot2$in_or_n) / nrow(plot2)
int_val2 <- area * prop_in2
plot_g_2 <- plot_val(plot2, int_val2)

#Code for plot 3
plot3 <- get_lst(1000, 0, 20, 0, 0.16, func_chisq)
plot3 <- data.frame(
  x = sapply(plot3, function(p) p[1]),
  y = sapply(plot3, function(p) p[2]),
  in_or_n = sapply(plot3, function(p) p[3])
)
area = 20 * 0.16
prop_in3 <- sum(plot3$in_or_n) / nrow(plot3)
int_val3 <- area * prop_in3
plot_g_3 <- plot_val(plot3, int_val3)
```

```r
#Code for plot 4
plot4 <- get_lst(10000, 0, 20, 0, 0.16, func_chisq)
plot4 <- data.frame(
  x = sapply(plot4, function(p) p[1]),
  y = sapply(plot4, function(p) p[2]),
  in_or_n = sapply(plot4, function(p) p[3])
)
area = 20 * 0.16
prop_in4 <- sum(plot4$in_or_n) / nrow(plot4)
int_val4 <- area * prop_in2
plot_g_4 <- plot_val(plot4, int_val4)

plot_g_1 <- plot_g_1 + labs(title = "n = 10")
plot_g_2 <- plot_g_2 + labs(title = "n = 100")
plot_g_3 <- plot_g_3 + labs(title = "n = 1,000")
plot_g_4 <- plot_g_4 + labs(title = "n = 10,000")

plot_g_1 + plot_g_2 + plot_g_3 + plot_g_4
library(dplyr)

# 1. Read data
df <- read.csv("calcium.csv", header = TRUE)

# 2. Remove final row
df <- df[-nrow(df), ]

# 3. Rename columns so they are unique
names(df) <- c("y0_a","y1_a","y2_a","y3_a",
               "y0_b","y1_b","y2_b","y3_b")

# 4. Compute the average for each year for each person
df_clean <- df %>%
  mutate(
    `0` = (y0_a + y0_b) / 2,
    `1` = (y1_a + y1_b) / 2,
    `2` = (y2_a + y2_b) / 2,
    `3` = (y3_a + y3_b) / 2
  ) %>%
  select(`0`, `1`, `2`, `3`) %>%     # keep only the year columns
  mutate(across(everything(), ~ round(.x, 1)))  # round to 1 decimal

# 5. Ensure trailing .0 is preserved
df_clean <- df_clean %>%
```

```r
  mutate(across(everything(), ~ format(.x, nsmall = 1)))

df_clean
library(tidyverse)

# --- Tidy the data (using the code previously written) ---

df <- read.csv("calcium.csv", header = TRUE)

# Give unique names to the repeated columns
colnames(df) <- c("y0_a","y1_a","y2_a","y3_a",
                  "y0_b","y1_b","y2_b","y3_b")

# Add person ID
df <- df %>% mutate(Person = row_number())

# Tidy into long format
tidy <- df %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = c("Year","Group"),
    names_pattern = "y(\\d)_(.)",
    values_to = "Value"
  ) %>%
  mutate(
    Year = as.integer(Year),
    Group = ifelse(Group == "a", "A", "B")
  ) %>%
  arrange(Person, Group, Year)

# --- Plot the data ---

ggplot(tidy, aes(x = Year, y = Value, group = Person, color =
 ↪  factor(Person))) +
  geom_line(linewidth = 1) +
  facet_wrap(~ Group, ncol = 1) +
  scale_x_continuous(breaks = 0:3) +
  scale_y_continuous(
    limits = c(min(tidy$Value, na.rm = TRUE),
               max(tidy$Value, na.rm = TRUE))
  ) +
  labs(
    title = "Calcium Levels Over Time",
```

```
    x = "Year",
    y = "Calcium Level",
    color = "Person"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "right",
    strip.text = element_text(size = 14, face = "bold")
  )
library(tidyverse)

df <- read.csv("calcium.csv", header = TRUE)

# Remove final row
df <- df[-nrow(df), ]

# Name columns
colnames(df) <- c("A_y0","A_y1","A_y2","A_y3",
                  "B_y0","B_y1","B_y2","B_y3")

# Add person ID
df <- df %>% mutate(Person = row_number())

# Round all numeric values
df <- df %>% mutate(across(starts_with(c("A","B")), ~round(.x, 1)))

# Reorder columns
df <- df %>% select(Person, everything())

df
library(tidyverse)

# ---- Load and clean data ----
df <- read.csv("calcium.csv", header = TRUE)

# Remove final row
df <- df[-nrow(df), ]

# Assign column names
colnames(df) <- c("A_y0","A_y1","A_y2","A_y3",
                  "B_y0","B_y1","B_y2","B_y3")

# Add person ID
```

```r
df <- df %>% mutate(Person = row_number())

# Round numeric values
df <- df %>% mutate(across(starts_with(c("A","B")), ~round(.x, 1)))

# ---- Convert to long format for plotting ----
long <- df %>%
  pivot_longer(
    cols = c(A_y0:A_y3, B_y0:B_y3),
    names_to = c("Calcium_Level", "Year"),
    names_pattern = "([AB])_y(\\d)",
    values_to = "Value"
  ) %>%
  mutate(
    Calcium_Level = ifelse(Calcium_Level == "A", "Calcium Level A", "Calcium
     ↪  Level B"),
    Year = as.numeric(Year)
  )

# ---- Plot Calcium Level A ----
plot_A <- long %>%
  filter(Calcium_Level == "Calcium Level A") %>%
  ggplot(aes(x = Year, y = Value, group = Person, color = factor(Person))) +
  geom_line(size = 1) +
  labs(title = "Calcium Level A Over Time",
       x = "Year",
       y = "Calcium Level",
       color = "Person") +
  theme_minimal()

# ---- Plot Calcium Level B ----
plot_B <- long %>%
  filter(Calcium_Level == "Calcium Level B") %>%
  ggplot(aes(x = Year, y = Value, group = Person, color = factor(Person))) +
  geom_line(size = 1) +
  labs(title = "Calcium Level B Over Time",
       x = "Year",
       y = "Calcium Level",
       color = "Person") +
  theme_minimal()

# Print plots
plot_A
```

```r
plot_B
##AIRPORTS CODE
library(tidyverse)
library(rvest)
library(dplyr)
library(knitr)

web = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_t
 ↪  raffic#2025_statistics"

airports_list <- web %>%
  read_html() %>%
  html_elements("table.wikitable") %>%
  html_table(fill = TRUE)

##Save the tables to individual variables by year
data_2025 = airports_list[[1]]
data_2024 = airports_list[[2]]
data_2023 = airports_list[[3]]
data_2022 = airports_list[[4]]
data_2021 = airports_list[[5]]
data_2020 = airports_list[[6]]

##Rename variables
rename_var <- function(in_data) {
  in_data %>%
    rename(
      "Total Passengers" = Totalpassengers,
      "Rank Change" = Rankchange,
      "Percent Change" = "%change"
    )
}

##Apply the function
data_2025 <- rename_var(data_2025)
data_2024 <- rename_var(data_2024)
data_2023 <- rename_var(data_2023)
data_2022 <- rename_var(data_2022)
data_2021 <- rename_var(data_2021)
data_2020 <- rename_var(data_2020)

##Remove Footnotes and brackets from numbers and convert it to an integer
pass_to_int <- function(in_data){
```

```
  in_data %>%
    mutate(
      "Total Passengers" = .data[["Total Passengers"]] %>%
        gsub("\\[.*?\\]", "", .) %>%
        gsub("[^0-9]", "", .) %>%
        as.integer()
    )
}

##Apply the function
data_2025 <- pass_to_int(data_2025)
data_2024 <- pass_to_int(data_2024)
data_2023 <- pass_to_int(data_2023)
data_2022 <- pass_to_int(data_2022)
data_2021 <- pass_to_int(data_2021)
data_2020 <- pass_to_int(data_2020)

##Add a "Year" column
data_2025$Year <- 2025
data_2024$Year <- 2024
data_2023$Year <- 2023
data_2022$Year <- 2022
data_2021$Year <- 2021
data_2020$Year <- 2020

##Make a master table, only getting the airports we want
master_table <- bind_rows(data_2025, data_2024, data_2023, data_2022,
 ↪  data_2021, data_2020) %>%
  filter(Airport %in% c("Hartsfield-Jackson Atlanta International Airport",
                        "Frankfurt Airport",
                        "Beijing Daxing International Airport",
                        "Dubai International Airport",
                        "Dallas Fort Worth International Airport",
                        "Heathrow Airport")) %>%
  select(Year, Airport, "Total Passengers")

master_table <- master_table %>%
  mutate(`Total Passengers` = round(`Total Passengers` / 1000000))

kable(master_table, caption = "Annual Passenger Volume of Airports
 ↪  (2020-2025, in Millions)")

ggplot(master_table, aes(x = Year, y = `Total Passengers`, color = Airport))
 ↪  + geom_point(size = 3) +
```

```
  geom_line() +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Annual Passeneger Traffic Trends (2020-2025)"
  )


##MONTE CARLO CODE
library(tidyverse)
library(rvest)
library(dplyr)
library(roxygen2)
library(patchwork)


#'Make a function that generates a random list of 2 numbers
#'
#' @description generates a random point for x and y coords
#'
#' @param minx Minimum x value
#' @param maxx Maximum x value
#' @param miny Minimum y value
#' @param maxy Minimum y value
#'
#' @return A vector in the format (x,y)
coord_gen <- function(minx, maxx, miny, maxy){
  x_point <- runif(1, min = minx, max = maxx)
  y_point <- runif(1, min = miny, max = maxy)
  return(c(x_point, y_point))
}

#'Make a function that runs the generating function n times and saves the
 ↪  results to a list
#' @description runs the coord_gen function n times
#'
#' @param n Number of times to generate a set of coords
#' @param minx Minimum x value
#' @param maxx Maximum x value
#' @param miny Minimum y value
#' @param maxy Minimum y value
#'
#' @return A list of vectors in the format (x,y)
gen_coords <- function(n, minx, maxx, miny, maxy){
```

```
  i = 1
  ret_lst = list()
  while(i<=n){
    ret_lst[[i]] <- coord_gen(minx, maxx, miny, maxy)
    i = i + 1
  }
  return(ret_lst)
}

#'Check to see if the points are above or below the function
#'
#' @description Checks to see if a given coord pair is in a function
#'
#' @param in_point The vector at which to evaluate
#' @param func The function which is used to check if the point is in or not
#'
#' @return TRUE if in, FALSE otherwise
check_in <- function(in_point, func)
  {
  if(in_point[[2]] <= func(in_point[[1]])){
    return(TRUE)
  }
  return(FALSE)
}

#'Append the list with the result of check_in
#'
#' @description Adds the result of check_in to elements in a list
#'
#' @param in_lst The list to be evaluated
#' @param func The function to pass to check_in
#'
#' @return A list containing the new in_or_n row
add_check <- function(in_lst, func){
  for(i in 1:length(in_lst)){
    in_or_n <- check_in(in_lst[[i]], func)
    in_lst[[i]] <- c(in_lst[[i]], in_or_n)
  }
  return(in_lst)
}

#'Put all the previous functions together to get the final list
#'
```

```r
#' @description Runs all previous functions to compile a list to be converted
 ↪  to a dataframe
#'
#' @param n Number of times to generate a set of coords
#' @param minx Minimum x value
#' @param maxx Maximum x value
#' @param miny Minimum y value
#' @param maxy Minimum y value
#' @param func The function to be passed to add_check
#'
#' @return A list with 3 columns of x value, y value, and in_or_n
get_lst <- function(n, minx, maxx, miny, maxy, func){
  first_lst <- gen_coords(n, minx, maxx, miny, maxy)
  new_lst <- add_check(first_lst, func)
  return(new_lst)
}

#'Define the Chi-Square function
#'
#' @description Defines the chi-square function
#'
#' @param x The value to be evaluated using the PDF of the chi-square
 ↪  function
#'
#' @return A y value of the position of a point along the chi-square function
func_chisq <- function(x) {
  return(dchisq(x, df = 5))
}

#'Plot the data
#'
#' @description Graphs the chi-square function and the point generated in the
 ↪  code
#'
#' @return The dot plot of x,y pairs
plot_val <- function(result_frame, int_val){
  ggplot(result_frame,
         aes(x = x, y = y, color = in_or_n)) +
    geom_point() +
    stat_function(
      fun = dchisq,
      args = list(df = 5),
      xlim = c(0, 20),
```

```
      color = "green"
    ) +
    annotate("text",
             x = 15, y = -0.03,
             label = paste("Est. Numerical Integration: ", int_val))
 }

 ##Code for plot 1
plot1 <- get_lst(10, 0, 20, 0, 0.16, func_chisq)
plot1 <- data.frame(
   x = sapply(plot1, function(p) p[1]),
   y = sapply(plot1, function(p) p[2]),
   in_or_n = sapply(plot1, function(p) p[3])
 )
area = 20 * 0.16
prop_in1 <- sum(plot1$in_or_n) / nrow(plot1)
int_val1 <- area * prop_in1
plot_g_1 <- plot_val(plot1, int_val1)

#Code for plot 2
plot2 <- get_lst(100, 0, 20, 0, 0.16, func_chisq)
plot2 <- data.frame(
  x = sapply(plot2, function(p) p[1]),
  y = sapply(plot2, function(p) p[2]),
  in_or_n = sapply(plot2, function(p) p[3])
)
area = 20 * 0.16
prop_in2 <- sum(plot2$in_or_n) / nrow(plot2)
int_val2 <- area * prop_in2
plot_g_2 <- plot_val(plot2, int_val2)

#Code for plot 3
plot3 <- get_lst(1000, 0, 20, 0, 0.16, func_chisq)
plot3 <- data.frame(
  x = sapply(plot3, function(p) p[1]),
  y = sapply(plot3, function(p) p[2]),
  in_or_n = sapply(plot3, function(p) p[3])
)
area = 20 * 0.16
prop_in3 <- sum(plot3$in_or_n) / nrow(plot3)
int_val3 <- area * prop_in3
plot_g_3 <- plot_val(plot3, int_val3)
```

```r
#Code for plot 4
plot4 <- get_lst(10000, 0, 20, 0, 0.16, func_chisq)
plot4 <- data.frame(
  x = sapply(plot4, function(p) p[1]),
  y = sapply(plot4, function(p) p[2]),
  in_or_n = sapply(plot4, function(p) p[3])
)
area = 20 * 0.16
prop_in4 <- sum(plot4$in_or_n) / nrow(plot4)
int_val4 <- area * prop_in2
plot_g_4 <- plot_val(plot4, int_val4)

plot_g_1 <- plot_g_1 + labs(title = "n = 10")
plot_g_2 <- plot_g_2 + labs(title = "n = 100")
plot_g_3 <- plot_g_3 + labs(title = "n = 1,000")
plot_g_4 <- plot_g_4 + labs(title = "n = 10,000")

plot_g_1 + plot_g_2 + plot_g_3 + plot_g_4




##PROMPTING GENAI CODE
##Prompted
library(tidyverse)

# --- Tidy the data (using the code previously written) ---

df <- read.csv("calcium.csv", header = TRUE)

# Give unique names to the repeated columns
colnames(df) <- c("y0_a","y1_a","y2_a","y3_a",
                  "y0_b","y1_b","y2_b","y3_b")

# Add person ID
df <- df %>% mutate(Person = row_number())

# Tidy into long format
tidy <- df %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = c("Year","Group"),
    names_pattern = "y(\\d)_(.)",
```

```
    values_to = "Value"
  ) %>%
  mutate(
    Year = as.integer(Year),
    Group = ifelse(Group == "a", "A", "B")
  ) %>%
  arrange(Person, Group, Year)

# --- Plot the data ---

ggplot(tidy, aes(x = Year, y = Value, group = Person, color =
 ↪  factor(Person))) +
  geom_line(linewidth = 1) +
  facet_wrap(~ Group, ncol = 1) +
  scale_x_continuous(breaks = 0:3) +
  scale_y_continuous(
    limits = c(min(tidy$Value, na.rm = TRUE),
               max(tidy$Value, na.rm = TRUE))
  ) +
  labs(
    title = "Calcium Levels Over Time",
    x = "Year",
    y = "Calcium Level",
    color = "Person"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "right",
    strip.text = element_text(size = 14, face = "bold")
  )


##General
library(tidyverse)

# ---- Load and clean data ----
df <- read.csv("calcium.csv", header = TRUE)

# Remove final row
df <- df[-nrow(df), ]

# Assign column names
colnames(df) <- c("A_y0","A_y1","A_y2","A_y3",
```

```
                  "B_y0","B_y1","B_y2","B_y3")

# Add person ID
df <- df %>% mutate(Person = row_number())

# Round numeric values
df <- df %>% mutate(across(starts_with(c("A","B")), ~round(.x, 1)))

# ---- Convert to long format for plotting ----
long <- df %>%
  pivot_longer(
    cols = c(A_y0:A_y3, B_y0:B_y3),
    names_to = c("Calcium_Level", "Year"),
    names_pattern = "([AB])_y(\\d)",
    values_to = "Value"
  ) %>%
  mutate(
    Calcium_Level = ifelse(Calcium_Level == "A", "Calcium Level A", "Calcium
     ↪  Level B"),
    Year = as.numeric(Year)
  )

# ---- Plot Calcium Level A ----
plot_A <- long %>%
  filter(Calcium_Level == "Calcium Level A") %>%
  ggplot(aes(x = Year, y = Value, group = Person, color = factor(Person))) +
  geom_line(size = 1) +
  labs(title = "Calcium Level A Over Time",
       x = "Year",
       y = "Calcium Level",
       color = "Person") +
  theme_minimal()

# ---- Plot Calcium Level B ----
plot_B <- long %>%
  filter(Calcium_Level == "Calcium Level B") %>%
  ggplot(aes(x = Year, y = Value, group = Person, color = factor(Person))) +
  geom_line(size = 1) +
  labs(title = "Calcium Level B Over Time",
       x = "Year",
       y = "Calcium Level",
       color = "Person") +
  theme_minimal()
```

```
# Print plots
plot_A
plot_B
```